Day 8 (Morning Session)

Express.js: Request Body, Middleware, express.json() and the Concept of an API

Lecture notes - Instructor: Abhishek Purohit - Date: 28 Oct 2026 - Notes for: Sumeet Sharma

#	Topic covered in this lecture
1	Limits of URL parameters and why we need the request body
2	Request body: the concept, GET vs POST, Postman and fetch
3	Reading the request body on the server: why request.body is undefined by default
4	Middleware: concept, flow, the next argument, multiple middlewares
5	Middleware demo (with and without next())
6	Reusing middleware on many routes (reference, not a call) and app.use()
7	express.json(): a function that returns a middleware; reading x and y from the body
8	Assignments: factorial, calculate-bill, circle
9	API (Application Programming Interface): meaning, examples, benefits
10	Questions answered in class
11	Revision: summary, definitions, differences, mistakes, interview Q&A, exercises, checklist

How to read these notes
Blue boxes hold the instructor's explanation in clear written English. Yellow boxes are additional explanations added by me; they were **not** said in the lecture. Red boxes list common mistakes. Every program from your class notes is included with explanation and expected output. This lecture did not cover the DOM, events or forms, so those are not part of these notes. Class chatter and breaks are left out.

1. Limits of URL Parameters

In the last session data was sent on the URL and the server received it through `request.params`. That approach has limitations:

Limitation	Explanation from the lecture
Not for sensitive data	Passwords, banking details, OTPs or any secret details must not be maintained on the URL.
Character limit	Only about **a thousand characters** can be maintained on a URL; you cannot send more than that.
Not suitable for structured data	If you want to send an **array** or a **JSON** to the server, the URL is not preferable.

In all those cases we use a new way of sending data to a server: the **request body**.

2. The Request Body

2.1 The concept

Instructor's explanation
Recall **response body**: a response has discrete elements, the status, the header and the body (the response data). In the same way a **request** has discrete elements: it contains the **URL**, the **method**, and it can also contain a **body**. Whenever a client sends a request it can attach data to it. That data is **not visible on the URL**; it travels with the request, and we call it the **request body**. The **response body** is the data a server sends to a client; the **request body** is the exact reverse (the instructor called it 'uno reverse'): the data a **client sends to the server**.

The request body can be JSON, an array, form data, a file, or other types of data.

	Direction	Example
Response body	Server to client	{ "message": "Hello world" }
Request body	Client to server	{ "x": 5, "y": 2 }
URL parameters	Client to server, on the URL (visible)	/sum/3/5

2.2 The contract between back end and front end

The client (front end) defines the request body in code, for example with the **fetch** function. The server must be programmed to read it. The back-end developers decide the **keys** that the request body must contain and tell the front-end developers (through documentation): this is the URL to send the request to, and you must send a request body with these keys. The **values can differ, but the keys must remain exactly the same** for all requests.

2.3 GET versus POST

Common mistake
With the **GET** method you cannot work with a request body. To send data in a request body the client must send a **POST** request (the method that is compatible with the request body).

2.4 Sending a request body with Postman

1. Choose the request method **POST** and mention the URL (for example http://127.0.0.1:6500/add).
2. Open the **Body** section (the other sections are params, authorization, headers).

3. Choose **raw**. Other options are form data, URL encoded data and binary (for a file); GraphQL is also there.
4. From the data type drop-down choose **JSON**. In raw you could also send a string, JavaScript, HTML or XML, but real applications nowadays use **JSON**: the server gives JSON and the client sends JSON.
5. Type the JSON in the input box and click Send.

```
{
    "x": 5,
    "y": 2
}
```

We use Postman because a back-end developer should only test the back-end functionality and does not need to build a front end for it. In real projects the front-end people send the post request and the request body programmatically.

2.5 Sending a request body from the front end with fetch

The first argument of fetch is the URL. Without more information fetch sends a GET request, so the second argument is a **configuration object** where you mention the method as POST and the **body** key. The key name is **body**.

```
// client side (front end)
fetch("http://127.0.0.1:6500/sum", {
    method: "POST",
    body: { x: 3, y: 7 }
});
```

Additional note (not from the lecture)
This is the form in your class notes. In a real browser the body must be a string and the server needs to know it is JSON, so you normally write `headers: { "Content-Type": "application/json" }` and `body: JSON.stringify({ x: 3, y: 7 })`. Without the Content-Type header, express.json() (below) will not parse the body. The instructor said the fetch function and the axios library would be used later.

A browser's address bar cannot send a POST with a body; that is why tools such as Postman, Thunder Client or similar exist. From a browser you would need a front end sending the request with fetch.

3. Reading the Request Body on the Server

Just as the URL parameters are in `request.params`, the request body is in **request.body**. The first server code simply logs it and sends a dummy response, to see what arrives:

```
app.post("/add", (request, response) => {
    console.log(request.body);                      // what do we get?
    response.send({ message: "add response" });
});
```

Run: node index.js; send a POST on **/add** in Postman with the JSON body { "x": 6, "y": 2 }.

```
Output:
Postman response:  { "message": "add response" }
Server console:    undefined
```

Common mistake
request.body is undefined even though the client sent a body. It is not the fault of the client; the server is not reading it.

Instructor's explanation
By default the Express server does **not process the request body**; the request.body property is always undefined. The reason: the request body can contain many different types of data: text, JSON, XML, HTML, JavaScript code and more. Express cannot decide by itself which type you want, so the decision is left to the developer. It is turned off by default; you must configure it explicitly to deal with the specific type you expect (JSON, XML, text and so on). Only then will your server read that type of data.

To configure this we need a new concept: **middleware**.

4. Middleware

4.1 What is a middleware?

Instructor's explanation
A middleware is a function that is executed **in the middle**: it is called **before the processing of the request**, and it can decide whether the request should be processed or not. Currently, as soon as a request arrives, processing starts with the callback that has the request and response arguments. With a middleware, in between: the request arrives, Express first calls the middleware, and only then (if allowed) does the request go to the processing function.

Flow	Sequence
Without middleware	request arrived -> (request, response) callback
With middleware	request arrived -> middleware -> (request, response) callback

The middleware acts like a **gate**: it holds the control and decides when the gate opens and when it stays closed. Typical uses:

- Check whether the user is logged in (authentication) or whether the user has a subscription needed for the resource.
- Check keys, security conditions and other filters.
- If the request should not proceed, the middleware itself sends a response (for example: you are not eligible / not logged in).
- **Modify the request object** (add more properties) before the processing starts.

4.2 How to define and place a middleware

Instructor's explanation
In a route you write the URL, then the middleware function, then the request/response processing function: `app.get(url, middleware, (req, res) => {})`. You may pass **any number of middlewares on a single route**: m1, m2, m3. Each performs its own validation (for example m1: is the user logged in, m2: is the user subscribed, m3: other check) and any middleware can stop the processing at any point by sending a response. If m1 allows, m2 is called; if m2 allows, m3 is called; when m3 allows, the request is finally processed.

```
app.get(url, (req, res, next) => { /* middleware */ }, (req, res) => { /* processing */ });

app.get(url, m1, m2, m3, (req, res) => { /* processing */ });   // many middlewares
```

4.3 The three arguments: request, response, next

Instructor's explanation
A middleware function also receives the **request** and **response**; it can read the request and, if something is fishy or missing, send a response itself. A **third argument, next**, holds the **reference of the next function**: for m1 it is m2, for m2 it is m3, and for the last middleware it is the final request/response function. **Calling next() passes control to the next function.** If you do not call next(), the request will not start processing.

4.4 Middleware demo

Purpose: see the middleware being called before the processing.

Version 1: the middleware does not call next()

```
app.get("/middleware-demo", (req, res, next) => {
    console.log("control is in middleware...");
    // next() is NOT called
}, (req, res) => {
    console.log("req started processing...");
    res.send({ message: "Dummy response" });
});

Output:
Server console:  control is in middleware...
Postman:         "Sending request..." (it just waits; no response ever comes)
```

Why: the middleware did not allow the next function to run, so the processing function was never executed and no response was generated.

Version 2: the middleware calls next()

```
app.get("/middleware-demo", (req, res, next) => {
    console.log("control is in middleware...");
    next();                                  // allow the request to continue
}, (req, res) => {
    console.log("req started processing...");
    res.send({ message: "Dummy response" });
});

Output:
Server console:
control is in middleware...
req started processing...

Postman:  { "message": "Dummy response" }
```

Restart the server after each code change (Ctrl + C, then node index.js).

Handling a failed check (student question): inside the middleware write an **if / else**: if the user is authenticated, call **next()**; else send a response such as { message: "user is not authenticated" } (with an appropriate status) and do not call next().

Additional note (not from the lecture)
The instructor described this in words. A sketch: `if (authenticated) { next(); } else { res.status(401).send({ message: "user is not authenticated" }); }` (401 is the standard status for 'not authenticated'; the instructor did not name a code).

5. Reusing a Middleware

5.1 Same middleware on several routes: pass the reference

If five or six routes need the same check (authentication or subscription), writing the same function on every route is a duplicate: if the logic changes later you must update it in many places. That solution works but is not the right approach. Instead define the function once, give it a name, and pass its **reference** to every route that needs it.

> **Instructor's explanation**
> Do we pass the **reference** or **call** the function? The **reference**, not a call. We are not going to call the middleware function ourselves: **Express calls it**. We only pass the definition (the reference), so we write `myMiddleware` and not `myMiddleware()`.

```
const myMiddleware = (req, res, next) => {
    console.log("control is in middleware...");
    next();
};

app.get("/", myMiddleware, (request, response) => {
    response.status(200).send({ message: "Hello world" });
});

app.get("/login", myMiddleware, (request, response) => {
    response.send("welcome to login");
});

// /sum/3/5
app.get("/sum/:a/:b", myMiddleware, (request, response) => {
    const paramsData = request.params;                    // { a: "3", b: "5" }
    var z = parseInt(paramsData.a) + parseInt(paramsData.b);
    response.send({ sum: z });
});
```

Naming: every middleware gets a different name that describes its job, for example **authenticateUser** for the login check and **checkSubscription** for the subscription check. **Sequence matters**: first authenticate the user and only then check the subscription, because without an authenticated user there is no user data to check.

5.2 Same middleware on every route: app.use()

> **Instructor's explanation**
> What if the middleware is needed on **each and every route** (say 200 routes)? Solution 1: mention the reference on every route (200 times). A better solution: call **app.use(myMiddleware)** once. The **use** function (called on the express application reference) makes every route of the application run that middleware first. Register it **before the routes**; you can then remove the reference from the individual routes. Routes developed later also get the middleware automatically.

```
const app = express();

app.use(myMiddleware);          // runs first for EVERY route

app.get("/", (request, response) => {
    response.status(200).send({ message: "Hello world" });
});
// ...no need to mention myMiddleware on each route

Output:
Console on a request to / :        control is in middleware...
Postman:                           { "message": "Hello world" }
(same console message appears for every route, for example /login and /sum/3/5)
```

If one route should not use the middleware (for example 99 routes need it and one does not), or if you need different middlewares for different groups of routes: you cannot simply override with app.use. If a route also has its own middleware (for example m2), the app-level middleware (myMiddleware) runs first and then m2. For grouping routes the instructor said you will use the **router object**, to be learnt later: one group of routes with the middleware and another group without it.

Instructor's explanation
There is no limitation on what a middleware can do: it can read the request, process data, work with a database or send a response. But follow **separation of concerns**: do not write your business logic or request processing inside the middleware. Use it for things like checking authenticity and verifying subscriptions.

6. express.json(): Processing a JSON Request Body

Instructor's explanation
We saw that Express does not read the body unless told which type to process. Express gives a ready-made middleware for JSON, reachable through the **express** reference: **express.json()**. Important: **express.json is itself not a middleware function**. Calling **express.json() returns a middleware function** that parses the JSON data in the request body. This is possible in JavaScript because a function can return another function.

6.1 On a single route

```
const jsonMiddleware = express.json();     // returns a middleware function

app.post("/add", jsonMiddleware, (request, response) => {
    console.log(request.body);             // no longer undefined
    response.send({ message: "add response" });
});
```

Postman: POST on /add with raw JSON body { "x": 8, "y": 2 }. The server console now prints the object instead of undefined.

```
Output:
Server console:   { x: 8, y: 2 }
```

6.2 For the whole application (recommended)

Instructor's explanation
An application may have hundreds of routes that work with a request body. Writing express.json() on each route is wasteful. Write it **once with app.use**: `app.use(express.json());` and every route is enabled to read a JSON request body (GET routes simply ignore it because GET does not carry a body). In real applications we always go with this option.

6.3 Using the request body: the add route

Purpose: add two numbers that the client sends in the request body.

```
import express from "express";
const app = express();

app.use(express.json());                        // process JSON request bodies

app.post("/add", (request, response) => {
    const { x, y } = request.body;              // destructure the keys x and y
    const z = x + y;
    response.send({ sum: z });
});

app.listen(6500);
```

Explanation: request.body is the JSON object sent by the client. Destructuring takes out the keys **x** and **y**; z adds them; the sum is sent as JSON. The keys in Postman must be exactly x and y, otherwise the destructured values will be undefined and you will get errors or NaN.

```
Output:
POST /add   body: { "x": 5, "y": 2 }   ->   { "sum": 7 }
```

Instructor's explanation
Notice that you did not need **parseInt** here. With a request body, the **data type is persisted**: a number sent by the client arrives as a number. With URL parameters the value always arrives as a string.

Point	URL parameters	Request body
Where the data is	On the URL (visible)	Inside the request, not on the URL
Read from	`request.params`	`request.body`
Needs configuration?	No	Yes: `app.use(express.json())`
Data types	Always strings (use parseInt)	Types persisted (numbers stay numbers)
Sensitive data / size	Not suitable; about 1000 characters	Suitable for sensitive and larger data (arrays, JSON)
Methods	Any, typically GET	POST (not with GET)

Complete index.js of the lecture (reference style), from your class notes with the URL parameter and body routes together:

```
import express from "express";
// const express = require("express");

const app = express(); // it will create an express application
// and this express application is nothing but your server

//127.0.0.1:6500/

const myMiddleware = (req, res, next) => {
    console.log("control is in middleware...");
    next();
};

app.get("/", myMiddleware, (request, response) => {
    response.status(200).send({ message: "Hello world" });
});

app.get("/login", myMiddleware, (request, response) => {
    response.send("welcome to login");
});

// /sum/3/5
app.get("/sum/:a/:b", myMiddleware, (request, response) => {
    // express prepares an object of all the url parameters: {a:3, b:5}
    const paramsData = request.params; // url params
    var z = parseInt(paramsData.a) + parseInt(paramsData.b);
    response.send({ sum: z });
});

app.get("/factorial/:num", (request, response) => {
    const n = parseInt(request.params.num); // request.params = {num:4}
    if (n < 0) {
        response.status(400).send({ message: "Negative values are not allowed" });
    } else {
        let fact = 1;
```

```
        for (var i = 1; i <= n; i++) {
            fact = fact * i;
        }
        response.send({ factorial: fact });
    }
});

app.post("/add", (request, response) => {
    console.log(request.body);
    response.send({ message: "add response" });
});

app.get("/middleware-demo", myMiddleware, (req, res) => {
    console.log("req started processing...");
    res.send({ message: "Dummy response" });
});

app.listen(6500);
```

Additional note (not from the lecture)
This is the file from your notes (the spelling 'allowerd' in the message is corrected). Note that the /add route here still logs request.body: add `app.use(express.json());` before the routes and use the destructuring version above to make it return the sum.

7. Assignments Given

Implement these routes using the request body (solutions are not included; test with Postman and the POST method):

Route	Request body	Response / rules
Factorial	The number in the request body	Factorial of the number (as before, think about invalid input)
/calculate-bill	`{ rate: 766, qty: 7 }` (rate of the product and quantity purchased)	total amount, discount applied, net amount. Discount slabs on the total: **3000 to 7000: 7%**; **7001 to 15000: 10%**; **above 15000: 15%** (flat)
/circle	`{ radius: ... }`	area and perimeter (circumference) of the circle

Additional note (not from the lecture)
The instructor did not say what discount applies below 3000; decide that rule yourself (for example no discount) and state it in your response. The transcript of the slab description is partly garbled; the slabs above follow your class notes.

8. API: Application Programming Interface

The routes we created are in fact **APIs**: the server receives a request (possibly with JSON data) and sends JSON in the response; this is how real servers work (similar to the JSON placeholder server used earlier, whose JSON data you fetched). Now let us understand the term. The full form is **Application Programming Interface**. Start with **interface** (not the Java interface).

8.1 What is an interface?

Instructor's explanation
An interface is the **middle person** between two things. **ATM:** between you and the ATM machine (cash machine) the bank provides a user interface, which lets you deposit or withdraw cash. **Mobile phone:** the phone has camera, speaker and SIM hardware, but to make a call, record video or play audio you use software, the user interface (the camera application, the phone application). The interface sits between the hardware and the user.

8.2 What is an API?

Instructor's explanation
An **API** is the interface that lets you use the **functionalities or services of one application inside another application**. It looks like a confusing term but the idea is simple: for using the functionalities of one application into another, we make use of an API. An API provides the functionality of one application to another.

Example: booking a flight

To book an Indigo flight from Indore to Mumbai you can use the Indigo website, the Indigo mobile app (Android or iOS), or third-party websites such as MakeMyTrip, Goibibo or Yatra. All of them can book the same flight.

Instructor's explanation
This is possible because of APIs. Indigo does not give the entire system or database to others. It officially provides **APIs** to search for flights, book a ticket, cancel a ticket and update a booking. MakeMyTrip, Goibibo and Yatra call Indigo's API URLs, sending a request body in JSON, and get JSON in the response. So one application (MakeMyTrip) uses features and functionalities that were developed by another application (Indigo).

Our routes are likewise APIs, for example /book-ticket, /search-flight, /cancel-ticket, /update-booking. Other examples: IRCTC / Rail One, RedBus and others using railway APIs; hundreds of websites that check **PNR**

status simply call a railway API, send the PNR number in the URL or request body, and get the status in the response, then build their own interface on top. You can also build your own applications (even a competitor of MakeMyTrip) using such APIs.

8.3 Benefits of building a back end as APIs

Benefit	Explanation
Reuse of functionality	One application's features can be used inside another application.
Front end and back end are independent	The development cycle is completely separated: front end and back end can be developed in parallel. The back end works only on routes, request body, response body, database connectivity and queries; it does not depend on the front end (React, Angular or Vue).
Back-end technology is hidden	The front end only sees the API URL, the request and the JSON response. If the back end changes from Node.js to Java (or Python), nothing changes at the front end as long as routes, requests and responses stay the same.
One back end for many platforms	A website, an Android app and an iOS app can all use the same APIs.

Instructor's explanation
Example: a customer wants a social media application with a website, an Android app and an iOS app. The website uses HTML, CSS, JavaScript (or React); the back end could be Java with MySQL. For the Android app you write the Android front end (UI) and for iOS the iOS front end (UI), but you do **not** rebuild the back end or the database. You create the back end once as APIs (login, logout, sign up, get profile data, get posts, save a comment, post a photo) and every front end consumes those APIs. YouTube, Flipkart, Amazon and WhatsApp work this way: separate interfaces (web, Android, iOS; the button components differ by platform) but one back end and one database behind them.

Next session: a real-time API with database connections: saving records into a database and getting data from a real database. Until then practise request bodies using the three assignments.

9. Questions Answered in Class

Question	Instructor's answer
How do we show a POST in the browser?	A browser address bar cannot send a POST with a body; you need a front end that uses fetch. That is why tools like Postman, Thunder Client and others exist.
What if the middleware check fails (user not authenticated)?	Write a condition: if the user is authenticated call next(); in the else block send a response with a JSON message such as 'user is not authenticated'.
With app.use applied to every route, how do I use a different middleware for one route?	A route-level middleware runs after the app-level one; it cannot override it. To exclude one route or group routes differently, use the router object (later topic).
Is there any limitation for a middleware?	No limitation, but keep business logic out of it (separation of concerns).
Do I pass the middleware reference or call it?	Pass the reference; Express calls it.

10. Revision Material

10.1 Summary of all major topics

- **URL parameters** are not for sensitive data, limited to about 1000 characters and not suitable for arrays or JSON; use the **request body**.
- **Request body** = data the client sends to the server inside the request (reverse of the response body). Not possible with GET; use POST. Keys are decided by back-end developers.
- Postman: POST, Body, raw, JSON. Front end: fetch(url, { method, body }).
- By default Express does not process the body, so **request.body is undefined**; you must configure the type of data to process.
- **Middleware** = function called before the request is processed; it can allow or stop the request, filter, authenticate, or modify the request. Signature (req, res, next).
- Calling **next()** passes control to the next function; without it the request hangs. Any number of middlewares can be placed on one route; sequence matters.
- Reuse a middleware by passing its **reference** (not a call). For all routes use **app.use(middleware)** before the routes. Router object (later) for exceptions.
- **express.json()** returns a middleware that parses a JSON body. Use **app.use(express.json())** once; read with **const { x, y } = request.body**. Data types are preserved.
- An **API** is an interface that lets one application use the services of another. Routes we create are APIs. Benefits: reuse, independent front end and back end, hidden back-end technology, one back end for web/Android/iOS.

10.2 Important definitions and syntax

Term / Syntax	Meaning
Request body	Data sent by the client inside the request (not on the URL)
Response body	Data sent by the server to the client
Middleware	Function executed before request processing; can allow or stop the request
next	Third argument of a middleware; calling next() passes control to the next function
`app.use(fn)`	Register a middleware for every route
`app.post(url, m1, m2, handler)`	POST route with middlewares before the handler
`express.json()`	Returns a middleware that parses JSON request bodies
`request.body`	The parsed request body (undefined until express.json() is used)
`const { x, y } = request.body;`	Destructure keys from the request body
`fetch(url, { method: "POST", body })`	Send a POST with a body from the front end
API	Application Programming Interface: interface to use one application's functionality in another
Interface	Middle person between two things (user and ATM, user and phone hardware)

10.3 Key differences between related concepts

Concept A	Concept B	Key difference
request.params	request.body	URL parameters (strings, visible) vs body data (types preserved, hidden, needs configuration)
Request body	Response body	Client to server vs server to client
GET	POST	No body vs can carry a body
express.json	express.json()	express.json is just a function; calling it returns the middleware function
Middleware reference	Middleware call	Pass myMiddleware (Express calls it); myMiddleware() would run it immediately
Middleware on a route	app.use(middleware)	One route (or listed routes) vs every route
Middleware	Route handler	Gate / checks / modifies the request vs final processing and response
Interface (Java)	API / user interface	A Java construct vs a middle layer between two systems or user and a machine

10.4 Common mistakes to avoid

Mistake	Why it fails	Correction
Reading request.body without express.json()	It is undefined	Add app.use(express.json())
Sending a body with GET	GET cannot carry a body	Use POST
Middleware without next()	The request hangs, no response	Call next() to continue, or send a response
Calling next() and also sending a response in a failed check	Both paths run	if ok, next(); else send a response only
Writing myMiddleware() on the route	Runs once at registration; not a function reference	Pass myMiddleware
Registering app.use after the routes	Routes defined earlier do not get it	Register before the routes
Different keys in Postman and in the destructuring	Values are undefined / NaN	Keep the keys exactly the same (x and y)
Not restarting the server after code changes	Old code keeps running	Ctrl + C, node index.js
Putting business logic in middleware	Breaks separation of concerns	Use middleware for checks (authentication, subscription)
Using parseInt on a JSON body number	Unnecessary; the type is already preserved	Use the values directly

10.5 Interview questions and answers

Items marked (extra) go beyond the lecture.

Question	Answer
Why use a request body instead of URL parameters?	Not for sensitive data, URL length is limited (about 1000 characters), and arrays/JSON are not suitable on a URL.
What is a request body?	The data a client sends to the server inside the request, not visible on the URL.

Question	Answer
Which HTTP method is used to send a body?	POST (GET cannot carry a body).
Why is request.body undefined by default?	Express does not process the body unless configured, because the body can be JSON, text, XML, HTML and so on.
What is a middleware?	A function executed before the request is processed; it can allow or stop the request, filter, or modify the request.
What are the arguments of a middleware?	request, response and next.
What does next() do?	Passes control to the next function; without it the request does not proceed.
Can we have more than one middleware on a route?	Yes, any number; they run in order.
How do you apply a middleware to every route?	app.use(middleware), registered before the routes.
Why pass a reference rather than call the middleware?	Express calls it when a request arrives; calling it yourself would run it immediately.
What does express.json() return?	A middleware function that parses JSON request bodies.
How do you read x and y from the request body?	const { x, y } = request.body; after app.use(express.json()).
Difference in data types between params and body?	Params are always strings; the body preserves types.
What is an API?	Application Programming Interface: an interface that lets one application use another application's functionality.
Why build back ends as APIs?	Reuse, independent front-end and back-end development, hidden back-end technology, a single back end for web, Android and iOS.
Give a real-world API example.	Indigo's flight APIs used by MakeMyTrip, Goibibo and Yatra; railway PNR status APIs.
How do you reject unauthenticated requests in a middleware?	If authenticated call next(); else send a response (for example a JSON message) without calling next().
(extra) What header must the client send for express.json() to parse the body?	Content-Type: application/json.

10.6 Practice exercises (solutions not included)

Easy

1. Explain in your own words the difference between request.params and request.body.
2. In Postman send a POST with JSON { "x": 5, "y": 2 } to a route that only logs request.body. What do you see before and after adding app.use(express.json())?
3. Write a middleware that logs a message and calls next(), and put it on one route.

Medium

1. Make the middleware run for every route using app.use and verify with two different routes.
2. Write /subtract that takes x and y from the request body and returns the difference. Do you need parseInt?
3. Create two middlewares and place both on one route; print messages to see the order.
4. Send a body with a wrong key name and describe what happens.

Challenging

1. Implement the three assignments: factorial with the number in the body, /calculate-bill with the discount slabs, /circle with area and perimeter.
2. Write a middleware that rejects a request with status 400 when the request body does not contain the key x, and calls next() otherwise.
3. Write authenticateUser and checkSubscription middlewares and put them on a route in the correct sequence. Why does the sequence matter?
4. Design the APIs (URL, method, request body keys, response) for a small booking application and write them as a document a front-end developer could use.

10.7 Final revision checklist

- [ ] I can list three limitations of URL parameters.
- [ ] I can explain the request body and why it needs POST.
- [ ] I can send a JSON body from Postman and describe the fetch call.
- [ ] I can explain why request.body is undefined by default.
- [ ] I can explain what a middleware is, with the flow and the next argument.
- [ ] I can write a middleware with and without next() and predict the output.
- [ ] I know to pass a middleware reference and not call it.
- [ ] I can use app.use() to apply a middleware to every route.
- [ ] I know express.json() returns a middleware and I can use app.use(express.json()).
- [ ] I can read keys from request.body by destructuring.
- [ ] I can explain why body data types are preserved while params are strings.
- [ ] I can explain what an API is, with the Indigo / MakeMyTrip example.
- [ ] I can list the benefits of building the back end as APIs.